Assignment Overview

- Objectives: You have joined the vision team of a Hsinchu startup that builds low-cost roadside cameras. The team wants to know how much it gains by moving from a hand-built neural network to a convolutional one. You will derive backpropagation by hand, connect neural networks to SVM and Bayesian learning, implement a neural network from scratch, and design a CNN.
- Constraints:
  1. Part II: NumPy only (no PyTorch, autograd or `scikit-learn` models). Part III: PyTorch, with no pretrained weights or external data.
  2. Hold out the last 5,000 training images as the validation set for all model selection, and evaluate on the test set only once. TA baseline accuracies on this split are announced on e3.
  3. All code must run on Google Colab (a free tier GPU, when available, is sufficient). Keep notebook outputs and fix all random seeds to `0`.
- General Instructions: The assignment consists of three compulsory components:
  1. Part I: Neurons by Hand (20 Points)
  2. Part II: A Neural Network from Scratch (45 Points)
  3. Part III: Convolutional Networks (35 Points)
- Submission Guidelines: Submit your work through the e3 submission system, including the following files:
  1. `A3_[ID].zip`: Your notebook(s) with all outputs and `nn_numpy.py`.
  2. `A3_[ID].pdf`: Your solutions to Part I, all requested tables and plots, your written analysis, and a viewable link to your Colab notebook.
- Late Submission Policy:
  1. `25% penalty`: From 15 minutes to 1 day late
  2. `50% penalty`: From 1 day to 2 days late
  3. `100% penalty`: More than 2 days late

Part I: Neurons by Hand [20 marks]

Before trusting a camera with thousands of images, every engineer traces one input through a tiny network by hand. Round all values to four decimal places and show your working.

1. The Toy Network. A 2–2–1 network has sigmoid hidden units, an identity output and the loss $J = \frac{1}{2}(y - \hat{y})^2$:

$$\mathbf{h} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad \hat{y} = \mathbf{W}^{(2)}\mathbf{h} + b^{(2)},$$

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.1 & 0.3 \\ 0.2 & 0.4 \end{bmatrix}, \ \mathbf{b}^{(1)} = \mathbf{0}, \ \mathbf{W}^{(2)} = \begin{bmatrix} 0.5 & -0.3 \end{bmatrix}, \ b^{(2)} = 0,$$

   with input $\mathbf{x} = (1.0,\, 0.5)^\top$ and target $y = 1$.

2. Tasks.

   (a) Architecture (4 points): A two-layer network has $\mathbf{W}^{(1)} \in \mathbb{R}^{K \times (S+1)}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{I \times (K+1)}$, with biases in column 0. Draw the network, including bias units, and count its parameters for CIFAR-10 images ($S = 3072$, $K = 100$, $I = 10$).

   (b) Backpropagation (8 points): For the toy network, compute the forward pass ($\mathbf{h}$, $\hat{y}$, J), all gradients via $\delta^{(2)} = \hat{y} - y$ and $\delta_i^{(1)} = \delta^{(2)} W_i^{(2)} \sigma'(z_i^{(1)})$, and one gradient descent step with $\lambda = 0.5$. Confirm that the loss decreases, and explain what goes wrong if all weights start at zero.

   (c) Loss Functions (4 points): For predicted probabilities $\hat{y} = [0.3, 0.2, 0.5]$ and label $y = [0, 1, 0]$, give the predicted class, the squared loss and the cross-entropy loss. For scores $s = [1.2, 1.8, 0.6]$ with true class 1, compute the multiclass hinge loss $\sum_{j \neq y} \max(0, s_j - s_y + 1)$ and $\partial L / \partial s$, and relate it to the SVM hinge loss.

   (d) Connections (4 points): With Gaussian noise $\mathcal{N}(0, \sigma^2)$ and prior $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \tau^2 \mathbf{I})$, show that MAP estimation equals L2-regularized least squares and find λ. Then explain whether a network with five linear hidden layers and a softmax output is more expressive than logistic regression.

Part II: A Neural Network from Scratch [45 marks]

The team's first prototype uses no deep learning library, so that every engineer understands what happens inside.

1. The Dataset: CIFAR-10. CIFAR-10[1] has 60,000 color images of size 32×32 in 10 classes, split into 50,000 training and 10,000 test images[2]. Download it with `torchvision.datasets.CIFAR10`[3], convert it to NumPy, scale pixel values to $[0, 1]$, flatten each image to 3072 values, and subtract the mean training image (computed on the 45,000 training images only).

2. The Module. Implement the following in `nn_numpy.py`, with each forward function returning a `cache` for its backward function:

Function	Description
`affine_forward/backward`	$Z = XW + b$ and gradients for X, W, b
`relu_forward/backward`, `sigmoid_forward/backward`	activations and their gradients
`softmax_ce_loss(S, y)`, `hinge_loss(S, y)`	mean loss over the batch and $\partial J/\partial S$
`numerical_gradient(f, x, h=1e-5)`	centered finite differences

3. Tasks.

   (a) Gradient Check (15 points): Vectorize all functions (no loops over examples). On a small random network in `float64`, report the relative error $\max_i \frac{|a_i - n_i|}{\max(10^{-8}, |a_i| + |n_i|)}$ between analytic and numerical gradients for every layer and both losses. Each should be below 10^{-6}.

   (b) Training (15 points): Train a two-layer `affine--ReLU--affine` network with L2 regularization and mini-batch SGD. First, overfit 50 images as a sanity check. Then tune the hidden size, learning rate, and regularization on the validation set, report at least four configurations in one table, and plot the loss and accuracy curves of your best model, which must beat the TA baseline (100 hidden units, weights drawn from $\mathcal{N}(0, 10^{-4})$, zero biases, no regularization, softmax loss, plain SGD with learning rate 10^{-3}, batch size 200, 10 epochs) for full marks. Finally, compare softmax and hinge loss as the output loss.

   (c) Design Issues (15 points): With five hidden layers, compare (i) zero, small (std $= 10^{-2}$), large (std $= 1$) and He (std $= \sqrt{2/\text{fan_in}}$) Gaussian initialization, (ii) sigmoid and ReLU under the same initialization, plotting the mean absolute gradient of each layer's weights at the start of training, and (iii) at least three L2 strengths. Explain each result using the design issues from lecture, such as vanishing gradients and overfitting.

[1] A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical Report, University of Toronto, 2009.
[2] https://www.cs.toronto.edu/~kriz/cifar.html
[3] https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.CIFAR10.html

Part III: Convolutional Networks [35 marks]

A fully connected network ignores the structure of images. You will now measure how much a CNN improves and test it on the test set once.

(a) From Pixels to Filters (5 points): Six $5 \times 5 \times 3$ filters with stride 1 and no padding are applied to a $32 \times 32 \times 3$ image. Give the output size and parameter count, compare them with a fully connected layer of the same output size, and explain why weight sharing helps.

(b) Your CNN (15 points): Design and train your own CNN. Report its layers (output shape and parameter count) in one table, justify your design choices, and plot its loss and accuracy curves. For full marks, it must beat the TA baseline CNN (`conv3x3(32)--ReLU--maxpool2x2--conv3x3(64)--ReLU--maxpool2x2--linear(10)` with padding 1, Adam, learning rate 10^{-3}, batch size 64, 10 epochs, no augmentation) on the validation set.

(c) Sensitivity Study (10 points): Vary one configuration (e.g., learning rate, depth, dropout, batch normalization or augmentation) over at least four values. Plot validation accuracy against it, identify the best value, and relate your findings to underfitting and overfitting.

(d) Final Evaluation (5 points): Evaluate your final model once on the test set. It must beat the TA baseline CNN's test accuracy for full marks. Compare it with Part II and briefly comment on whether your validation accuracy was a reliable estimate.